Console-Based Library Management System

Project Documentation — Developers Arena · Week 3: Java Programming Basics

Author: Mohammad Kaif Khan | Submission date: 30 August 2026
Repository: https://github.com/Md-Kaif534/week3-library-system



Fig. 1 — Sample session: adding books, borrowing, and viewing statistics

1. Project Overview & Objectives

This project is a command-line library management system built for the Week 3: Java Programming Basics assignment. It is written in plain Java (no external libraries) and demonstrates object-oriented design, collections, file-based persistence, and exception handling.

Objectives

- Apply core OOP principles: encapsulation, constructors, getters/setters, and a clean separation of responsibilities across classes.
- Use ArrayLists and Java Streams to manage and query collections of books and members.
- Persist application state to text files and reload it correctly on the next run.
- Handle predictable failure cases (missing book, missing member, unavailable book) with custom checked exceptions rather than generic errors.
- Validate every piece of console input before it reaches the business logic.

Project Description

A Java console application for managing library operations including book tracking, member management, and a borrowing system with file-based data persistence.

Features

- Add, remove, and search for books
- Register and manage library members
- Borrow and return books with due dates
- Calculate overdue fines
- File-based data persistence
- Comprehensive input validation
- Library statistics generation

Technologies Used

- Java 17+ (classes, interfaces, Streams, exceptions, java.time)
- File I/O (java.io, java.nio.file)
- Maven (optional build/run wrapper)
- Git for version control

2. Setup & Installation Instructions

Requirement: JDK 17 or later. Check with **java -version**.

Option A — Plain javac / java (no Maven needed)

```
git clone https://github.com/Md-Kaif534/week3-library-system.git
cd week3-library-system

# Compile
javac -d bin src/main/java/library/*.java

# Run
java -cp bin library.Main
```

Option B — Maven

```
mvn compile
mvn exec:java

# or build a runnable jar
mvn package
java -jar target/library-system.jar
```

Either way, the program creates **data/books.txt** and **data/members.txt** automatically on first run if they don't already exist.

Git commands used for version control

```
git init
git add -A
git commit -m "Week 3: Java Programming Basics - console library management system"
git branch -M main
git remote add origin https://github.com/Md-Kaif534/week3-library-system.git
git push -u origin main
```

3. Code Structure Explanation

```
week3-library-system/
|-- src/main/java/library/
|   |-- Main.java             # Console menu loop, Scanner input, validation
|   |-- Book.java             # Book entity: fields, behavior, file (de)serialization
|   |-- Member.java           # Member entity: fields, borrowed-book tracking
|   |-- Library.java         # Facade: owns the collections, all business logic
|   |-- FileHandler.java   # The only class touching books.txt / members.txt
|   |-- BookNotFoundException.java
|   |-- MemberNotFoundException.java
|   `-- BookUnavailableException.java
|-- src/main/resources/
|-- data/
|   |-- books.txt         # Persisted book catalog (pipe-delimited)
|   `-- members.txt     # Persisted member list (pipe-delimited)
|-- docs/         # Screenshots used in documentation
|-- README.md
|-- .gitignore
`-- pom.xml
```

Why this split?

- **Main** only talks to the user (prompts, menus, input validation) and never touches a file or collection directly.
- **Library** is a facade: it owns the book/member Lists and all business rules (loan periods, fines, duplicate checks).
- **FileHandler** is the only class that opens a file — swapping storage later means rewriting one class.
- **Book** / **Member** are plain entities with private fields and public getters/setters, each owning its own file serialization.
- Three small custom exceptions replace generic error strings with typed, catchable failures.

4. Screenshots of Working Application

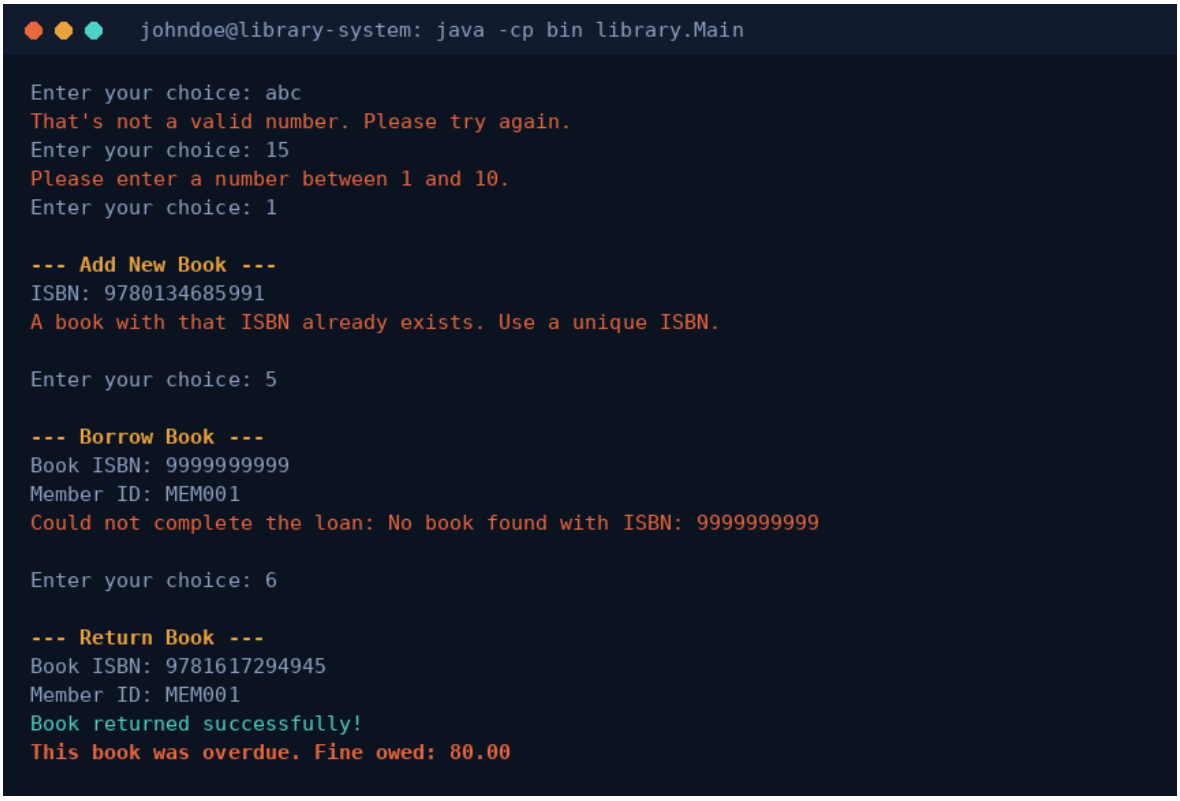


Fig. 2 — Input validation and exception handling: invalid menu choice, duplicate ISBN, unknown book/member, and an overdue return with a calculated fine

5. Technical Details

Class design (encapsulation)

Book and Member keep every field private; all access goes through getters/setters. Member.getBorrowedIsbns() returns an unmodifiable view — callers can read a member's borrowed books but must go through borrowBook()/returnBook() to change them, which keeps Book and Member records in sync.

Data structures

Books and members are stored in ArrayList<Book> / ArrayList<Member> inside Library, with ISBN/ID lookups done via a linear scan (fine at this scale). Each Member keeps its own List<String> of borrowed ISBNs so "what has this member borrowed" is a direct lookup rather than a scan over every book. Java Streams (filter/map/collect/count) power search, filtering, and statistics.

File I/O & persistence

Each Book/Member serializes itself to one pipe-delimited line and parses itself back, so the file format lives next to the class it describes. FileHandler wraps every read/write in try-with-resources plus try/catch(IOException), so a missing or unreadable file degrades to an empty list with a warning instead of crashing. A malformed line is skipped with a line-number warning rather than aborting the whole load. Every mutating Library method saves immediately, so at most the last action is lost on an unexpected exit.

Exception handling

Three checked exceptions — BookNotFoundException, MemberNotFoundException, BookUnavailableException — model the specific ways a borrow/return can fail. Main catches them at the point of use and prints the exception's own message. Menu input parsing is wrapped in try/catch(NumberFormatException) inside a loop, so a non-numeric choice just re-prompts.

Algorithms

Overdue fine — a flat linear model: ChronoUnit.DAYS.between(dueDate, today) * FINE_PER_DAY, computed lazily whenever asked for, so it's always correct relative to "now" rather than going stale. **Search** — case-insensitive substring match over title, author, and ISBN in a single Stream filter pass.

Requirements Mapping

Requirement	Implementation
Java classes with proper encapsulation	Book and Member expose only private fields with public getters/setters; Library hides its Lists behind methods that enforce business rules (no direct external mutation).
File I/O operations for data persistence	FileHandler reads/writes data/books.txt and data/members.txt as pipe-delimited text, wrapped in try-with-resources with IOException handling.

Requirement	Implementation
ArrayLists for storing collections	Library holds ArrayList<Book> and ArrayList<Member>; Member holds an ArrayList<String> of borrowed ISBNs.
Console-based menu system	Main runs a numbered menu loop (1-10) using Scanner, dispatching each choice via a switch expression.
Exception handling for errors	Three custom checked exceptions for domain errors, plus try/catch around all file I/O and numeric input parsing.
Input validation and user feedback	Dedicated helper methods re-prompt until a non-empty string, an in-range integer, or a valid-shaped email is entered; every action prints a clear success or failure message.
Search and filter functionality	Library.searchBooks()/searchMembers() filter by keyword across multiple fields using Streams.

6. Testing Evidence

Test case	Steps	Result
Add book	Choice 1, enter a new ISBN	Book added, confirmed, saved to books.txt — Pass
Duplicate ISBN	Choice 1, enter an existing ISBN	Rejected with a clear message — Pass
Invalid menu input	Enter "abc" or "15" at the menu	Re-prompted with a specific error, no crash — Pass
Register member	Choice 4, valid name/email	Member registered and saved — Pass
Invalid email	Choice 4, enter "not-an-email"	Rejected, re-prompted until valid — Pass
Borrow book	Choice 5, valid ISBN + member ID	Marked unavailable, due date set to +14 days — Pass
Borrow unknown book/member	Choice 5, bad ISBN or ID	Custom exception caught, friendly message — Pass
Borrow already-borrowed book	Choice 5 on a borrowed book	BookUnavailableException caught and named — Pass
Return + fine	Choice 6, on an overdue book	Made available again, fine computed and shown — Pass
Persistence across restart	Add data, exit, relaunch	"Loaded N books and M members" matches — Pass
Statistics	Choice 7	Totals, overdue count, and fines match current data — Pass
CSV export	Choice 9, a file path	Valid CSV written with all current book data — Pass
Malformed data line	Corrupt one line in books.txt, relaunch	Line skipped with a warning; rest of file loads — Pass

Contact

- Email: mdkaifkhan139@gmail.com
- GitHub: https://github.com/Md-Kaif534
- LinkedIn: https://www.linkedin.com/in/md-kaif-khan-3530252a5/